

Newtonsoft serialize `System.Drawing.Rectangle` thành CHUỖI, không phải object — port sang kiểu riêng là vỡ dữ liệu cũ

Mục	Nội dung
---	---
Ngày	2026-07-26
Project	DoorAlarmv3 (migrate WinForms → Avalonia)
Mức độ	HIGH — âm thầm, không exception
Từ khoá	Newtonsoft, JsonConvert, System.Drawing.Rectangle, TypeConverter, migrate, cross-platform

Bối cảnh

Bản WinForms lưu cấu hình vùng nhận diện camera vào `tblCamera.ConfigRegions` bằng `JsonConvert.SerializeObject(new CameraConfig { ConfigRegions = List<ConfigRegion> })`, trong đó `ConfigRegion.Rect` kiểu `System.Drawing.Rectangle`. Khi port sang Avalonia (cross-platform, bỏ `System.Drawing`), phải thay `Rectangle` bằng kiểu tự viết. Giả định tự nhiên là JSON có dạng:

```
{ "Rect": { "X": 1240, "Y": 880, "Width": 144, "Height": 192 } }
```

Điều thực tế xảy ra

Newtonsoft ưu tiên `TypeConverter` của kiểu nếu có. `System.Drawing.Rectangle` có `RectangleConverter` → serialize thành **chuỗi**:

```
{ "Rect": "1240, 880, 144, 192", "Name": "B1", "IsLocked": false, ... }
```

Kiểu mới (record struct 4 field int) deserialize chuỗi đó → **không lỗi, không exception**, chỉ ra `default` (0,0,0,0). Toàn bộ vùng cấu hình cũ biến mất, màn hình hiện "chưa có vùng nào", và AI detection không bao giờ bật. Không có gì trong log để lần ra.

Cách phát hiện

Đọc thẳng dữ liệu thật trước khi viết model — đừng suy ra định dạng từ khai báo C#:

```
SELECT TOP 3 Id, DATALENGTH(ConfigRegions) AS L,  
LEFT(CAST(ConfigRegions AS nvarchar(max)), 500) AS J  
FROM tblCamera WHERE DATALENGTH(ConfigRegions) > 2;
```

(Cột `text` không so sánh được với `varchar` bằng `<>` — dùng `DATALENGTH` thay cho `<>` `'`.)

Cách khắc phục

Viết `JsonConverter<T>` giữ đúng định dạng chuỗi, kèm nhánh dự phòng đọc cả dạng object:

```
public sealed class RegionRectJsonConverter : JsonConverter<RegionRect>  
{  
    public override void WriteJson(JsonWriter w, RegionRect v, JsonSerializer s)  
        => w.WriteValue($"{{v.X}, {v.Y}, {v.Width}, {v.Height}}");  
}
```

```
public override RegionRect ReadJson(JsonReader r, Type t, RegionRect e, bool has,
JsonSerializer s)
{
    if (r.TokenType == JsonToken.String)
    {
        var p = ((string?)r.Value ?? "").Split(',', StringSplitOptions.TrimEntries);
        return p.Length == 4 && int.TryParse(p[0], out var x) && int.TryParse(p[1],
out var y)
            && int.TryParse(p[2], out var wd) && int.TryParse(p[3], out var h)
            ? new RegionRect(x, y, wd, h) : default;
    }
    // ... nhánh StartObject cho dữ liệu do phiên bản khác ghi
}
}
```

Rồi kiểm chứng round-trip bằng probe console chạy trên **CSDL thật**, không phải dữ liệu tự bịa:

```
ImageSize = 1920x1080, regions = 22
re-serialize: {"ImageWidth":1920,...,"ConfigRegions":[{"Rect":"1240, 880, 144, 192",...
[PASS] doc/ghi dung dinh dang WinForms
```

Nguyên tắc rút ra

1. Kiểu .NET có `TypeConverter` thì Newtonsoft dùng nó, không serialize theo property.

Nhóm hay dính: `Rectangle`, `Point`, `Size`, `Color`, `Guid`, `TimeSpan`, `Version`.

2. Khi migrate mà **hai bản dùng chung một CSDL**, định dạng serialize là **hợp đồng**, không phải chi tiết nội bộ. Đối kiểu C# ở một bên là phá hợp đồng.

3. Deserialize sai kiểu thường **im lặng** — luôn viết probe đọc dữ liệu thật và assert số lượng bản ghi + giá trị field đầu tiên, đừng chỉ tin "build sạch, app chạy".

Liên quan

- [\[\[avalonia-12-breaking-changes-rabbitmq7-migration\]\]](#)
- [\[\[winforms-to-avalonia-drawing-replacement\]\]](#)